RA1 · Práctica guiada de Nginx con PHP y Python

Instalar un servidor web en tu Raspberry Pi y demostrar cómo responde con HTML, PHP y Python.

0614 · Desplegament d'Aplicacions Web · 2.º DAW · 2026–2027

Qué vas a conseguir

Al terminar, desde el navegador de tu portátil abrirás tres páginas de tu Raspberry: una página HTML, una página PHP y una aplicación Python. Entenderás qué programa responde en cada caso y podrás repetir las pruebas después de reiniciar.

Antes de empezar, comprueba que la Raspberry está encendida, tiene Raspberry Pi OS Lite instalado, está conectada a la red del aula y dispone de una IP fija. Ten a mano el usuario y su contraseña. El docente debe confirmar los datos de acceso y que SSH esté habilitado. Necesitarás conexión a Internet para descargar los paquetes. Utilizaremos los paquetes de Debian/Raspberry Pi OS. Si has instalado otro servidor web o tienes aplicaciones del aula en ese equipo, consulta con el docente antes de cambiar su configuración.

Cómo utilizar el manual

Sigue los pasos en orden. Escribe una orden cada vez y pulsa Intro. Espera a que vuelva el indicador de la terminal, llamado prompt, antes de escribir la siguiente. Suele mostrar tu usuario y el nombre del equipo y terminar en $. No escribas ese indicador: escribe solo la orden. Los bloques oscuros contienen órdenes para la terminal. Los bloques claros contienen archivos que debes escribir en un editor o salidas de ejemplo; su etiqueta indica cuál de los dos casos corresponde.

Todas las órdenes Linux se ejecutan dentro de la Raspberry por SSH. Las órdenes de PowerShell se ejecutan en tu portátil. La dirección 127.0.0.1 siempre representa la máquina donde se realiza la prueba: en SSH es la Raspberry; en el navegador de Windows sería el portátil.

IP_ASIGNADA, USUARIO, SOCKET_PHP y SERVICIO_PHP son etiquetas: debes sustituirlas por tus valores antes de usar una plantilla. Por ejemplo, http://IP_ASIGNADA/ se convierte en http://192.168.1.45/ solo si esa es la IP fija asignada a tu equipo. No necesitas inventar un dominio ni configurar .local.

Prepara tus datos y los dos archivos

En el portátil abre el Explorador de archivos con Windows+E. En Documentos crea una carpeta llamada RA1-Nginx: pulsa Nuevo → Carpeta, escribe el nombre y confirma con Intro.

Abre el Bloc de notas desde Inicio. Escribe de momento tu nombre, el usuario de la Raspberry y la IP fija. Pulsa Archivo → Guardar como, entra en RA1-Nginx, elige Todos los archivos, escribe inventario-nginx.md como nombre y guarda en UTF-8. Comprueba que no se haya añadido .txt al final.

Crea un archivo nuevo en el Bloc de notas y guárdalo en esa carpeta como comprobaciones-nginx.txt. Puedes dejarlo vacío por ahora. La extensión .md permite usar títulos y listas en texto; puedes seguir editando ambos archivos con el Bloc de notas. Mantén esa carpeta accesible mientras realizas la práctica.

Tu entrega

Guardarás dos archivos en el portátil: inventario-nginx.md y comprobaciones-nginx.txt. En el capítulo 13 encontrarás sus plantillas. No debes redactar una memoria de todos los capítulos. Las páginas web y su configuración quedarán en la Raspberry para demostrar que funcionan.

Cómo guardar una comprobación

En Windows Terminal, selecciona con el ratón las líneas que quieres conservar y pulsa Ctrl+Mayús+C para copiarlas. En el Bloc de notas pulsa Ctrl+V y guarda con Ctrl+S. Escribe encima el comando utilizado y debajo una frase con el resultado. Si utilizas la ventana clásica de PowerShell, selecciona el texto y pulsa Intro para copiarlo.

No pulses Ctrl+C para copiar cuando una orden esté funcionando: en la terminal esa combinación puede detenerla. No copies páginas enteras de salida si solo necesitas un dato. Por ejemplo, para una prueba HTTP guarda la línea de estado y el texto que identifica la página.



Accede con tu usuario y la IP fija. Los nombres y las direcciones de este esquema son ejemplos; el acceso SSH utiliza el puerto 22 y la web utilizará el puerto 80.

1 · Entender qué es Nginx

Qué hace un servidor web

Cuando escribes una URL, el navegador envía una petición HTTP. Un servidor web la recibe y devuelve una respuesta: por ejemplo, el contenido de una página y un código de estado. Nginx se pronuncia «engine ex». Puede entregar archivos y también recibir peticiones para pasarlas a otros programas.

Una URL es la dirección completa de un recurso, por ejemplo http://192.168.1.45/prueba.php. http indica el protocolo; la IP identifica la Raspberry y /prueba.php indica qué página solicitas. Un puerto identifica la entrada de un servicio: SSH utiliza normalmente el 22 y HTTP el 80. Al escribir http:// sin indicar otro puerto, el navegador utiliza el 80.

HTML es el contenido que interpreta el navegador. PHP y Python son lenguajes que pueden generar ese contenido en el servidor. Nginx no ejecuta por sí mismo el código PHP o Python: necesita comunicarse con el programa que lo ejecuta.



Las tres rutas de esta práctica. El navegador entra por Nginx en el puerto 80. Nginx lee HTML, consulta PHP-FPM o pasa la petición a Gunicorn.

Reconoce cada pieza

Nginx recibe las peticiones web. PHP-FPM ejecuta los archivos PHP y devuelve su resultado mediante FastCGI. Flask nos permite escribir una aplicación pequeña en Python. Gunicorn ejecuta esa aplicación y atiende las peticiones HTTP que le pasa Nginx.

Un proxy inverso es un servidor situado delante de una aplicación. El navegador habla con Nginx, y Nginx habla con Gunicorn. En nuestra práctica, Gunicorn escuchará solo en 127.0.0.1:8000 dentro de la Raspberry; entrarás desde tu portátil por el puerto 80 de Nginx.

Comprueba que lo entiendes

Antes de instalar, responde oralmente: ¿quién devuelve el archivo HTML?, ¿quién ejecuta PHP?, ¿quién ejecuta la aplicación Python? Puedes volver a este esquema cuando una prueba falle.

2 · Preparar la sesión y los datos

2.1. Abre PowerShell y conecta

En tu portátil pulsa Inicio, escribe Terminal y abre Windows Terminal. En el menú de la flecha junto a las pestañas selecciona Windows PowerShell. También puedes buscar Windows PowerShell directamente en Inicio. No necesitas abrirlo como administrador.

Verás una línea que comienza normalmente por PS y termina en >. Ese indicador significa que ahora escribes en el portátil. Prepara la orden siguiente sustituyendo USUARIO y IP_ASIGNADA, sin espacios alrededor de @. Si tu usuario fuera alumne07 y tu IP asignada fuera 192.168.1.45, escribirías ssh alumne07@192.168.1.45. Utiliza tus propios datos:

En Windows · PowerShell del portátil

```
ssh USUARIO@IP_ASIGNADA
```

Pulsa Intro. Si aparece Are you sure you want to continue connecting, la terminal pregunta si confías en la clave de ese equipo. Antes de contestar, compara la huella SHA256 mostrada con la de tu Raspberry. En su consola con monitor y teclado se obtiene con la orden que aparece a continuación; el docente puede mostrarla si no tienes esa consola.

En la Raspberry · consola con monitor y teclado

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

La comparación debe corresponder al mismo tipo de clave. Si el aviso muestra otro tipo, pide ayuda para obtener esa huella concreta. Cuando coincida, vuelve al portátil, escribe yes y pulsa Intro. Después escribe la contraseña del usuario de la Raspberry y pulsa Intro; no verás puntos ni asteriscos mientras la escribes.

Si aparece Permission denied, revisa usuario y contraseña. Si aparece Connection timed out o Connection refused, comprueba con el docente que la IP sea correcta, que la Raspberry haya arrancado y que SSH esté habilitado. No continúes con las órdenes Linux hasta que la conexión esté abierta.

2.2. Reconoce la sesión de la Raspberry

Tras entrar por SSH, el indicador dejará de ser PS y normalmente mostrará algo como alumne07@daw-rpi-07:~ $. Los nombres pueden ser distintos: deben corresponder a tu equipo. El teclado sigue siendo el del portátil, pero ahora las órdenes se ejecutan en la Raspberry.

Ejecuta las siguientes órdenes una por una. No copies las cuatro como una única línea:

En la Raspberry · sesión SSH

```
hostname
whoami
ip -br a
cat /etc/os-release
```

Guarda hostname, usuario, IP fija y versión del sistema en inventario-nginx.md. Utilizaremos la IP que ya configuraste; esta práctica no cambia los datos de red.

hostname muestra el nombre del equipo; whoami, el usuario de la sesión. En ip -br a busca la IP fija junto a la interfaz Ethernet o Wi-Fi; 127.0.0.1 pertenece a la interfaz interna y no es la IP que usas desde Windows. En /etc/os-release busca PRETTY_NAME y VERSION_ID para anotar el sistema y su versión.

Si quieres abrir una terminal del portátil sin cerrar SSH, crea otra pestaña de PowerShell. Si escribes exit dentro de SSH, cerrarás esa sesión y volverás al portátil; la Raspberry seguirá encendida.

2.3. Comprueba el puerto antes de instalar

En la Raspberry · sesión SSH

```
sudo ss -ltnp
```

Busca una dirección terminada en :80. Si ya hay un proceso escuchando, identifica su nombre. Si es Apache u otro servicio, no lo detengas sin saber para qué se utiliza: acuerda con el docente cómo preparar el equipo. Si Nginx ya está instalado, revisa sus sitios existentes antes de continuar.

En la salida de ss, Local Address:Port es la dirección y el puerto de escucha; Process identifica el proceso. Una línea con 0.0.0.0:80 o [::]:80 indica que alguien utiliza ese puerto. Si no aparece ninguna línea terminada en :80, puedes continuar con la instalación. Ver una línea de :22 es normal: corresponde a SSH.

Si una orden pide una contraseña o abre un visor

Cuando sudo pida la contraseña, escríbela y pulsa Intro. Aunque no veas caracteres, se está recibiendo. Algunos comandos abren un visor que muestra (END) y no devuelve el prompt: pulsa q para salir de ese visor. No apagues la Raspberry para recuperar la terminal.

Si sudo solicita una contraseña, utiliza la de tu usuario de la Raspberry. sudo permite realizar tareas administrativas. Los comandos sin sudo se ejecutan con tu usuario normal.

3 · Instalar Nginx y comprobar su primera respuesta

3.1. Instala los paquetes

En la Raspberry · sesión SSH

```
sudo apt update
sudo apt install nginx curl nano
```

apt update actualiza el catálogo de paquetes. No continúes si muestra errores de DNS o descarga. apt install instala Nginx, curl y nano; revisa lo que propone y acepta con Y o S según el idioma. curl permite consultar una URL desde la terminal. nano permite editar archivos de texto.

Durante la instalación aparecerán mensajes de descarga y configuración. Espera a que termine y vuelva el prompt. Si APT pregunta Do you want to continue? [Y/n] o ¿Desea continuar? [S/n], escribe la letra afirmativa que muestra y pulsa Intro. Si has copiado una orden y aparece un error, conserva el mensaje y no escribas la siguiente hasta resolverlo.

3.2. Arranca y revisa el servicio

En la Raspberry · sesión SSH

```
sudo systemctl enable --now nginx
systemctl is-active nginx
sudo systemctl status nginx --no-pager
nginx -v
```

enable prepara el arranque automático; --now también inicia el servicio en este momento. is-active debe mostrar active. status proporciona más detalle y -v muestra la versión instalada. Un servicio activo todavía no demuestra que el navegador pueda acceder.

Lee la respuesta de is-active: active significa que el servicio está ejecutándose; inactive o failed indican que debes revisar status. En status busca Active: active (running). systemctl es la herramienta que gestiona servicios, es decir, programas que siguen trabajando en segundo plano aunque no tengas una ventana abierta.

3.3. Prueba desde la Raspberry

En la Raspberry · sesión SSH

```
curl -I http://127.0.0.1/
sudo ss -ltnp
```

curl -I pide las cabeceras. Debes encontrar un estado 200 y el puerto 80 en escucha. La versión HTTP mostrada puede variar. Esta prueba se realiza dentro de la Raspberry y verifica su respuesta local.

La I es mayúscula. Una cabecera HTTP contiene datos de la respuesta; el cuerpo contiene la página. Para esta prueba basta con localizar la línea de estado. Puede tener este aspecto:

Salida de ejemplo · no escribir en la terminal

```
HTTP/1.1 200 OK
Server: nginx
```

200 significa que la petición ha sido atendida correctamente. Server puede mostrar también una versión. No escribas esas líneas: son una salida de ejemplo.

3.4. Prueba desde Windows

En el portátil abre Edge, Chrome o tu navegador habitual. Haz clic en la barra de direcciones superior, escribe http:// seguido de tu IP fija y una barra final, y pulsa Intro. Por ejemplo, http://192.168.1.45/ solo si esa es tu IP. Utiliza la barra de direcciones, no el cuadro de búsqueda de una página. Debe aparecer la bienvenida de Nginx.

Para comprobar las cabeceras desde Windows, abre una nueva pestaña de PowerShell. No escribas esta orden en la pestaña donde has entrado por SSH. Sustituye IP_ASIGNADA y ejecuta:

En Windows · PowerShell del portátil

```
curl.exe -I http://IP_ASIGNADA/
```

Escribimos curl.exe para utilizar el programa curl de Windows y evitar confundirlo con un alias de PowerShell. Si la web funciona localmente y no desde Windows, revisa IP, red y filtrado del aula. No cambies PHP ni Python: todavía no intervienen.

Conserva tu primera prueba

Anota la versión de Nginx y guarda el estado del servicio y la cabecera HTTP recibida desde el portátil. En este momento solo has demostrado que Nginx responde.

4 · Crear tu primera página HTML

4.1. Prepara la carpeta de la web

En la Raspberry · sesión SSH

```
sudo mkdir -p /var/www/ra1-web
sudo chown "$USER":www-data /var/www/ra1-web
sudo chmod 755 /var/www/ra1-web
```

mkdir -p crea la carpeta si no existe. $USER toma el nombre de tu usuario actual; conserva el signo $ y las comillas. chown te permite escribir en esta carpeta y asigna el grupo www-data. chmod 755 permite que Nginx pueda atravesarla y leer el contenido. No uses permisos 777 para resolver errores.

Una ruta absoluta empieza por / y señala la misma ubicación desde cualquier carpeta. /var/www/ra1-web será la carpeta de nuestra web; index.html será un archivo dentro de ella. Si una orden termina sin mostrar texto y vuelve el prompt, puede haber funcionado correctamente. Lo comprobarás con ls, que lista archivos.

www-data es el usuario y grupo utilizado habitualmente por los servicios web de estos paquetes. 755 permite entrar en la carpeta y leer su listado; 644, que aplicarás a los archivos, permite leerlos. La escritura queda reservada a su propietario.

4.2. Crea index.html

Vas a utilizar nano, un editor de texto dentro de la terminal. Mientras estás en nano no se ejecutan comandos: lo que escribas pasará a formar parte del archivo.

En la Raspberry · sesión SSH

```
nano /var/www/ra1-web/index.html
```

Al abrir nano verás el nombre del archivo arriba y las combinaciones de teclas abajo. Si el archivo no existe, el área de escritura estará vacía. Copia solo el contenido del bloque siguiente, desde <!doctype html> hasta </html>; no copies el título Contenido del archivo. En Windows Terminal puedes pegar con Ctrl+Mayús+V. Si aparece un aviso de pegado de varias líneas, comprueba que estás dentro de nano antes de aceptar.

Conserva los signos < y > y las comillas. También puedes escribir el texto a mano:

Contenido del archivo · /var/www/ra1-web/index.html

```
<!doctype html>
<html lang="es">
<meta charset="utf-8">
<title>Mi servidor Nginx</title>
<h1>Mi Raspberry responde con Nginx</h1>
<p>Esta página es un archivo HTML.</p>
<p><a href="/prueba.php">Probar PHP</a></p>
<p><a href="/python/">Probar Python</a></p>
</html>
```

Guarda así: mantén Ctrl y pulsa la letra O, suelta ambas teclas y comprueba el nombre de archivo que aparece abajo. Pulsa Intro para confirmar. Después pulsa Ctrl+X para salir. Debes volver al indicador de la sesión SSH. Si al salir pregunta si quieres guardar, responde Y o S según lo que muestra y confirma con Intro.

En las ayudas de nano, ^O significa Ctrl+O y ^X significa Ctrl+X. Muévete con las flechas; usa Retroceso para borrar y Intro para crear una línea. Las dos páginas enlazadas todavía no existen: las crearás después. Puedes cambiar el título para reconocer tu trabajo.

En la Raspberry · sesión SSH

```
chmod 644 /var/www/ra1-web/index.html
ls -l /var/www/ra1-web/index.html
```

644 permite al propietario editar y a los demás leer. Comprueba que el archivo existe; crear un archivo en esa carpeta todavía no configura el sitio de Nginx.

Para revisar lo guardado sin abrir el editor, ejecuta cat /var/www/ra1-web/index.html. Debe mostrar el texto HTML que acabas de escribir. Si el archivo está vacío o falta parte del contenido, ábrelo otra vez con nano, corrígelo y repite el guardado. El resultado en el navegador se comprobará después de activar el sitio.

5 · Configurar y activar el sitio de la práctica

5.1. Entiende dónde se guarda la configuración

En los paquetes de Debian, sites-available contiene configuraciones guardadas; sites-enabled contiene enlaces a las que se utilizan. Activar un sitio consiste en crear su enlace. El archivo HTML y el archivo de configuración son distintos: uno contiene la página y el otro indica cómo servirla.

Un enlace es una referencia a otro archivo. En ls -l lo reconocerás por una flecha → o -> hacia su destino. Por ejemplo, default -> /etc/nginx/sites-available/default significa que Nginx utiliza el archivo default guardado en sites-available. Mantendrás ese archivo como referencia y activarás uno propio llamado ra1-web.

Antes de cambiar nada, revisa y guarda el sitio inicial:

En la Raspberry · sesión SSH

```
ls -l /etc/nginx/sites-enabled/
sudo cp -a /etc/nginx/sites-available/default \
  /etc/nginx/sites-available/default.ra1-backup
```

La barra invertida une las dos líneas en una sola orden Linux. No escribas espacios después de ella. Esta ruta presupone una instalación nueva desde APT con el sitio default. Si falta ese archivo o hay otros sitios, revisa la situación con el docente antes de continuar.

cp hace una copia; -a conserva los atributos. Aquí copia default a default.ra1-backup dentro de sites-available. Esta copia no activa otro sitio. Si repites la orden, sustituirá esa copia: comprueba si ya la tienes antes de repetirla. Para verla, ejecuta ls -l /etc/nginx/sites-available/default.ra1-backup.

5.2. Escribe el sitio

En la Raspberry · sesión SSH

```
sudo nano /etc/nginx/sites-available/ra1-web
```

Escribe el bloque completo. Conserva las llaves, los puntos y coma y los signos $: son parte de la configuración de Nginx.

Si el editor está vacío, pega el bloque. Si ra1-web ya contiene texto de otra sesión, revisa con el docente qué debes conservar antes de sustituirlo. Cada { abre un bloque y cada } lo cierra; la mayoría de las instrucciones terminan en ;. La sangría ayuda a reconocer qué queda dentro de server.

Después de escribir el contenido, pulsa Ctrl+O, Intro y Ctrl+X. Solo cuando hayas vuelto al prompt ejecuta las órdenes del siguiente apartado.

Contenido del archivo · /etc/nginx/sites-available/ra1-web

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
    root /var/www/ra1-web;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ \.php$ {
        return 404;
    }
}
```

listen indica el puerto. default_server elige este sitio cuando no hay otro nombre coincidente. server_name _ no crea un dominio ni un nombre de red. root señala la carpeta de documentos; index indica la página inicial. try_files busca el archivo pedido y devuelve 404 si no existe. El último bloque impide servir archivos PHP como texto mientras preparas PHP-FPM.

5.3. Activa y comprueba antes de aplicar

Solo en la Raspberry de esta práctica, sustituye el sitio inicial por tu sitio:

En la Raspberry · sesión SSH

```
sudo unlink /etc/nginx/sites-enabled/default
sudo ln -s /etc/nginx/sites-available/ra1-web \
  /etc/nginx/sites-enabled/ra1-web
sudo nginx -t
```

unlink retira el enlace del sitio inicial; el archivo original sigue guardado. ln -s crea el enlace a tu configuración. Si el enlace ra1-web ya existe, revísalo con ls -l y no repitas su creación. nginx -t debe indicar syntax is ok y test is successful. Si falla, corrige el archivo y repite la prueba. No recargues con errores.

Si nginx -t muestra un error, lee el archivo y el número de línea que cita. Abre ese archivo con sudo nano, busca la zona indicada y revisa llaves, puntos y coma y nombres. Puedes buscar texto con Ctrl+W, escribirlo y pulsar Intro. Guarda y sal; vuelve a ejecutar nginx -t. Solo continúa cuando aparezcan las dos confirmaciones de éxito.

En la Raspberry · sesión SSH

```
sudo systemctl reload nginx
curl http://127.0.0.1/
```

reload aplica la configuración sin detener el servicio. curl sin -I muestra el cuerpo HTML. Abre de nuevo http://IP_ASIGNADA/ en Windows: ahora debes ver tu página, no la bienvenida inicial. Actualiza el navegador con Ctrl+F5 si conserva la anterior.

Comprueba un archivo inexistente

En la Raspberry · sesión SSH

```
curl -I http://127.0.0.1/no-existe
```

Debe devolver 404. Esta respuesta indica que Nginx atiende la petición pero no encuentra ese recurso. Guárdala: no todos los códigos distintos de 200 significan que el servidor esté apagado.

6 · Instalar PHP-FPM y localizar su conexión

6.1. Instala PHP

En la Raspberry · sesión SSH

```
sudo apt install php-fpm php-cli
php -v
systemctl list-units --type=service --no-pager 'php*-fpm.service'
ls -l /run/php/
```

php-cli permite ejecutar PHP desde la terminal; PHP-FPM lo ejecutará para las peticiones web. La versión depende de los repositorios del sistema. Anota la que tienes, el nombre del servicio activo y la ruta del archivo terminado en -fpm.sock.

En la tabla de servicios, copia el nombre de la columna UNIT, incluido .service. En ls -l /run/php/ busca un nombre como php8.4-fpm.sock. Para formar la ruta completa, une la carpeta /run/php/ y ese nombre. Un archivo terminado en .pid no es el socket: no lo utilices en fastcgi_pass.

En tu inventario añade dos líneas separadas: Servicio PHP y Socket PHP. Usarás el servicio en systemctl y el socket en la configuración de Nginx. Copia los valores reales, no los del ejemplo.

Un socket Unix es un punto de comunicación entre procesos de la misma máquina. En esta práctica Nginx enviará allí las peticiones PHP; no es una URL para el navegador.

Ejemplo para interpretar tu salida

Salida de ejemplo · no escribir en la terminal

```
Servicio: php8.4-fpm.service
Socket:   /run/php/php8.4-fpm.sock
```

Estos valores son un ejemplo, no una versión que debas imponer. SERVICIO_PHP se sustituye por el nombre real del servicio y SOCKET_PHP por la ruta completa de su socket. Si hay varias versiones o no aparece un socket, detente y consulta qué versión corresponde a la práctica.

Por ejemplo, si tu servicio real es php8.4-fpm.service, la primera orden de abajo será sudo systemctl enable --now php8.4-fpm.service. Escribe el nombre como aparece en la tabla. Si se está iniciando, espera y vuelve a consultar /run/php/ antes de concluir que falta el socket.

En la Raspberry · sesión SSH

```
sudo systemctl enable --now SERVICIO_PHP
systemctl is-active SERVICIO_PHP
```

Sustituye la etiqueta en ambas órdenes. La respuesta debe ser active y el socket debe existir antes de configurar Nginx.

7 · Probar que PHP se ejecuta en el servidor

7.1. Escribe una página de prueba

En la Raspberry · sesión SSH

```
nano /var/www/ra1-web/prueba.php
```

Dentro de nano pega el contenido desde <?php hasta la última línea echo. El inicio <?php indica que el archivo contiene código PHP. Guarda con Ctrl+O, confirma la ruta con Intro y sal con Ctrl+X. Al volver al prompt, continúa con la comprobación de sintaxis:

Contenido del archivo · /var/www/ra1-web/prueba.php

```
<?php
header('Content-Type: text/html; charset=utf-8');
echo '<h1>PHP funciona en mi Raspberry</h1>';
echo '<p>Versión de PHP: ' . PHP_VERSION . '</p>';
echo '<p>Hora UTC del servidor: ';
echo gmdate('Y-m-d H:i:s') . '</p>';
echo '<p><a href="/">Volver al inicio</a></p>';
```

echo genera contenido; PHP_VERSION aporta la versión que ejecuta la página; gmdate obtiene la hora UTC al recibir la petición. La página no contiene información de configuración sensible ni utiliza phpinfo().

En la Raspberry · sesión SSH

```
chmod 644 /var/www/ra1-web/prueba.php
php -l /var/www/ra1-web/prueba.php
```

php -l comprueba la sintaxis y debe indicar que no hay errores. No demuestra todavía que Nginx se comunique con PHP-FPM.

7.2. Conecta Nginx con PHP-FPM

En la sesión SSH escribe sudo nano /etc/nginx/sites-available/ra1-web y pulsa Intro. Estás editando la configuración, no prueba.php.

Dentro de nano pulsa Ctrl+W, escribe return 404; y pulsa Intro. Debe llevarte a la línea del bloque location ~ \.php$. Sitúa el cursor en esa línea y pulsa Ctrl+K para cortar únicamente la línea return 404;. En su lugar escribe las dos instrucciones include y fastcgi_pass del bloque siguiente. Mantén la línea location y su llave de cierre. Si necesitas moverte o corregir, utiliza las flechas y Retroceso. El bloque PHP completo debe quedar así:

Contenido del archivo · Bloque PHP dentro de server

```
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:SOCKET_PHP;
    }
```

Reemplaza SOCKET_PHP por tu ruta completa. Con el socket del ejemplo, esa línea sería fastcgi_pass unix:/run/php/php8.4-fpm.sock;. Conserva unix: al principio y el punto y coma al final. El archivo incluido es el fragmento PHP que proporciona el paquete Debian de Nginx; no añadas otro bloque PHP ni modifiques ese fragmento.

Antes de guardar, compara la ruta que has escrito con la línea Socket PHP de tu inventario. No debe quedar la palabra SOCKET_PHP en el archivo. Guarda con Ctrl+O, Intro y Ctrl+X. El cambio está guardado, pero Nginx aún utiliza su configuración anterior hasta que la valides y recargues.

En la Raspberry · sesión SSH

```
sudo nginx -t
sudo systemctl reload nginx
curl -i http://127.0.0.1/prueba.php
```

Ejecuta reload solo después de que nginx -t termine correctamente. curl -i muestra cabeceras y cuerpo. Debes recibir 200 y el texto generado por PHP. No debe aparecer el código <?php en la respuesta. Si se descarga el archivo o aparece su código, deja de usar ese sitio y revisa la configuración PHP antes de continuar.

7.3. Prueba desde el portátil

Abre http://IP_ASIGNADA/prueba.php. Actualiza después de unos segundos: la hora UTC debe cambiar. Eso demuestra ejecución dinámica en el servidor, no solo un archivo leído por Nginx. Guarda el estado HTTP y la versión devuelta por la página; la versión de PHP CLI puede ser diferente si existen varias instalaciones.

Para obtener la cabecera desde Windows, abre tu pestaña local de PowerShell y ejecuta curl.exe -i http://IP_ASIGNADA/prueba.php con tu IP. Debajo del 200 verás etiquetas HTML: eso es el resultado generado, no el código PHP. En el navegador esas etiquetas se muestran como una página con título, versión y hora.

8 · Preparar una aplicación Python sencilla

8.1. Instala Python y crea su carpeta

En la Raspberry · sesión SSH

```
sudo apt install python3 python3-venv
python3 --version
sudo mkdir -p /opt/ra1-python
sudo chown "$USER":www-data /opt/ra1-python
sudo chmod 755 /opt/ra1-python
cd /opt/ra1-python
python3 -m venv .venv
```

La carpeta de Python está fuera de /var/www: su código no se ofrecerá como archivo público. cd cambia la carpeta actual. venv crea un entorno independiente en .venv para instalar las bibliotecas de esta aplicación sin modificar los paquetes Python del sistema.

pwd muestra la carpeta en la que estás. Ejecútalo al terminar cd: debe devolver /opt/ra1-python. Una ruta como .venv/bin/python no empieza por / y se interpreta desde esa carpeta. Si abres otra sesión SSH, vuelve a ejecutar cd /opt/ra1-python antes de usar órdenes que empiezan por .venv o nombres como app.py.

Comprueba que el entorno existe con ls -l .venv/bin/python. .venv es el nombre de una carpeta que empieza por punto; los listados normales pueden ocultarla. ls -a permite verla.

8.2. Instala Flask y Gunicorn en el entorno

En la Raspberry · sesión SSH

```
.venv/bin/python -m pip install Flask gunicorn
.venv/bin/python -m pip freeze > requirements.txt
```

Se utiliza el Python de .venv de forma explícita; no necesitas activar el entorno ni usar sudo pip. freeze guarda en requirements.txt las versiones instaladas para poder identificarlas después. El símbolo > escribe la salida en ese archivo y sustituye su contenido si existía. Si la descarga falla, revisa conexión y DNS.

pip es el instalador de bibliotecas de Python. El texto python -m pip indica que se utiliza el pip de ese intérprete concreto. Al terminar deben reaparecer el prompt y los mensajes de instalación, sin errores. Ejecuta cat requirements.txt para comprobar que el archivo contiene líneas con nombres y versiones. Flask permite construir la página; Gunicorn la atenderá como servicio web.

8.3. Escribe app.py

En la Raspberry · sesión SSH

```
nano /opt/ra1-python/app.py
```

Dentro de nano copia el bloque completo. Para las líneas de la función utiliza espacios, no tabuladores. La línea ahora y la línea return tienen cuatro espacios delante; las cadenas entre paréntesis tienen ocho. Python utiliza esa sangría para reconocer qué instrucciones pertenecen a una función. Guarda con Ctrl+O, Intro y Ctrl+X:

Contenido del archivo · /opt/ra1-python/app.py

```
from datetime import datetime, timezone
from flask import Flask

app = Flask(__name__)

@app.get('/')
def inicio():
    ahora = datetime.now(timezone.utc).isoformat()
    return (
        '<h1>Python funciona en mi Raspberry</h1>'
        f'<p>Hora UTC del servidor: {ahora}</p>'
        '<p><a href="/">Volver al inicio</a></p>'
    )
```

Flask relaciona la ruta / con la función inicio. La función genera HTML con la hora de cada petición. La variable app es la aplicación que cargará Gunicorn. No necesitas iniciar el servidor de desarrollo de Flask.

En la Raspberry · sesión SSH

```
.venv/bin/python -m py_compile app.py
chmod 644 app.py requirements.txt
```

Si py_compile termina sin errores, la sintaxis es válida. Si falla, revisa la línea indicada y la sangría. Esta comprobación no sustituye la prueba HTTP.

Si aparece IndentationError, revisa los espacios al principio de las líneas. Si aparece SyntaxError, revisa comillas y paréntesis en la línea indicada. Abre de nuevo nano /opt/ra1-python/app.py, corrige, guarda y repite py_compile hasta que vuelva el prompt sin mensajes de error.

9 · Probar Python antes de conectarlo con Nginx

9.1. Arranca Gunicorn temporalmente

En tu sesión SSH ejecuta cd /opt/ra1-python y después pwd. Cuando confirme esa carpeta, inicia Gunicorn con la siguiente orden. La barra invertida final une las dos líneas: puedes pegarlas juntas o escribir la primera, pulsar Intro y completar la segunda. Si aparece un indicador > de continuación, termina la orden; no copies ese indicador:

En la Raspberry · sesión SSH

```
.venv/bin/gunicorn --workers 1 \
  --bind 127.0.0.1:8000 --access-logfile - app:app
```

app:app significa «carga el archivo app.py y su variable app». --workers 1 crea un proceso de trabajo para esta prueba pequeña. --bind indica dónde escucha. --access-logfile - muestra las peticiones en la terminal. No se utiliza sudo: la aplicación se ejecuta con tu usuario.

Esta orden ocupa la terminal y no devuelve el prompt mientras funciona. Es normal. No cierres esa sesión todavía. 127.0.0.1 limita el acceso a la propia Raspberry; no debes abrir ese puerto a la red del aula.

9.2. Prueba desde una segunda sesión

Deja abierta la primera pestaña donde funciona Gunicorn. En Windows Terminal pulsa la flecha junto al botón + y abre otra pestaña de Windows PowerShell. En una ventana clásica puedes abrir una segunda ventana desde Inicio.

En la nueva pestaña escribe ssh USUARIO@IP_ASIGNADA con tus valores, pulsa Intro y autentícate. Comprueba que el indicador corresponde a la Raspberry. Ahora ejecuta estas órdenes en esa segunda sesión SSH; no necesitas cambiar de carpeta porque solo estás consultando una URL:

En la Raspberry · sesión SSH

```
curl -i http://127.0.0.1:8000/
sudo ss -ltnp
```

Debes recibir 200 y «Python funciona en mi Raspberry». En ss debe aparecer 127.0.0.1:8000. No pruebes esa dirección desde Windows: allí representaría tu portátil. Si no funciona esta prueba local, corrige Python antes de añadir el proxy de Nginx.

9.3. Detén la prueba temporal

Vuelve a la primera sesión y pulsa Ctrl+C. Espera a que vuelva el prompt. Has detenido Gunicorn; es normal que curl deje de responder en 8000. Ahora prepararás un servicio para que la aplicación no dependa de una terminal abierta.

10 · Mantener Python encendido con systemd

10.1. Crea el servicio

Consulta whoami y anota tu usuario exacto. Abre el archivo:

Escribe whoami en una terminal SSH que tenga el prompt disponible y copia exactamente el nombre que devuelve. La plantilla contiene una sola línea User=USUARIO. Si whoami devolviera alumne07, escribirías User=alumne07. No utilices User=root y no dejes la etiqueta USUARIO sin sustituir.

En la Raspberry · sesión SSH

```
sudo nano /etc/systemd/system/ra1-python.service
```

Escribe la plantilla sustituyendo USUARIO en User por el resultado de whoami. En ExecStart las líneas con barra invertida forman una misma orden; no añadas comentarios al final de esas líneas.

Contenido del archivo · /etc/systemd/system/ra1-python.service

```
[Unit]
Description=Aplicacion Python de la practica RA1
After=network.target

[Service]
User=USUARIO
Group=www-data
WorkingDirectory=/opt/ra1-python
ExecStart=/opt/ra1-python/.venv/bin/gunicorn \
    --workers 1 --bind 127.0.0.1:8000 \
    --access-logfile - app:app
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

User evita ejecutar la aplicación como root. WorkingDirectory indica dónde está app.py. ExecStart inicia Gunicorn usando el entorno de esa carpeta. Restart permite reintentar tras un fallo. WantedBy permite habilitar el servicio durante el arranque normal.

Conserva los títulos [Unit], [Service] y [Install], incluidos los corchetes. Revisa que las rutas coincidan con la carpeta y el entorno que has creado. Guarda con Ctrl+O, confirma /etc/systemd/system/ra1-python.service con Intro y sal con Ctrl+X. Si cambiaste el nombre del archivo, corrígelo antes de ejecutar enable.

10.2. Carga y prueba el servicio

En la Raspberry · sesión SSH

```
sudo systemctl daemon-reload
sudo systemctl enable --now ra1-python
systemctl is-active ra1-python
sudo systemctl status ra1-python --no-pager
curl -i http://127.0.0.1:8000/
```

daemon-reload hace que systemd vuelva a leer sus archivos de servicios; no recarga Nginx. enable --now habilita e inicia la aplicación. Debes recibir active y la misma respuesta Python. Si el puerto está ocupado, comprueba que detuviste la prueba temporal con Ctrl+C.

Cuando el servicio responda correctamente, puedes cerrar la pestaña donde hiciste la prueba temporal. Deja abierta una sesión SSH para continuar. La aplicación ya la mantiene systemd: no depende de una terminal con Gunicorn en primer plano.

Si no arranca

En la Raspberry · sesión SSH

```
sudo journalctl -u ra1-python -n 30 --no-pager
```

Busca el error concreto: usuario inexistente, ruta incorrecta, módulo que no se encuentra o puerto ocupado. Corrige el archivo o la aplicación. Si cambias el archivo .service, ejecuta daemon-reload y después sudo systemctl restart ra1-python. Si solo cambias app.py, basta con reiniciar ra1-python. Comprueba de nuevo curl.

11 · Conectar Nginx con la aplicación Python

11.1. Añade el proxy

Primero asegúrate de que curl http://127.0.0.1:8000/ devuelve la página Python. Después escribe sudo nano /etc/nginx/sites-available/ra1-web en la sesión SSH.

Muévete con las flechas hasta el final del archivo. Identifica la última llave }, que cierra server; no es la llave del bloque PHP. Coloca el cursor al principio de esa última línea, pulsa Intro para dejar una línea vacía antes de ella y vuelve con la flecha arriba a esa línea vacía. Pega allí los dos bloques siguientes. Deben quedar dentro de server y fuera de los bloques location existentes. Mantén el bloque PHP y una única llave final para server:

Contenido del archivo · Bloques Python dentro de server

```
    location = /python {
        return 301 /python/;
    }

    location /python/ {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
            $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
```

La directiva X-Forwarded-For ocupa dos líneas y termina con punto y coma. En este archivo no necesita una barra invertida para continuar; conserva el contenido tal como aparece.

Comprueba la distribución con el archivo completo de 11.2: un único bloque server contiene cuatro bloques location. No pegues el archivo completo debajo del que ya tienes. Utilízalo para comparar, conserva tu ruta real del socket PHP y guarda con Ctrl+O, Intro y Ctrl+X antes de salir al prompt.

La ruta /python se redirige a /python/. La barra final de proxy_pass hace que Nginx quite el prefijo /python/ al reenviar: Flask recibe /. Si omites esa barra, Flask recibiría /python/ y la aplicación de esta práctica no tendría esa ruta. Las cabeceras transmiten datos de la petición; no activamos confianza automática en ellas dentro de Flask.

11.2. Revisa el archivo completo

Al terminar, tu sitio debe tener esta estructura. Sustituye SOCKET_PHP por su ruta real y no dupliques el bloque server:

Contenido del archivo · /etc/nginx/sites-available/ra1-web completo

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
    root /var/www/ra1-web;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:SOCKET_PHP;
    }

    location = /python {
        return 301 /python/;
    }

    location /python/ {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
            $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Nginx permite que una directiva continúe en otra línea hasta su punto y coma. Los signos $ de este archivo son variables de Nginx: consérvalos; no son etiquetas que debes sustituir.

11.3. Valida, aplica y prueba

En la Raspberry · sesión SSH

```
sudo nginx -t
sudo systemctl reload nginx
curl -i http://127.0.0.1/python/
```

Recarga solo si la prueba de sintaxis es correcta. Ahora la respuesta debe ser 200 y mostrar la página Python a través del puerto 80 de Nginx. Gunicorn sigue escuchando únicamente en 127.0.0.1:8000.

Desde Windows abre http://IP_ASIGNADA/python/. Actualiza la página y comprueba que cambia la hora. El navegador no debe usar :8000. Desde la Raspberry puedes consultar también la redirección:

Para comprobar el recorrido, observa que la URL del navegador termina en /python/ y no incluye :8000. Ahora Nginx recibe la petición por 80 y la pasa a Gunicorn. El resultado debe coincidir con la prueba local de 8000, aunque la hora será la de la nueva petición.

En la Raspberry · sesión SSH

```
curl -I http://127.0.0.1/python
```

Debe responder 301 con una cabecera Location hacia /python/. Un 301 indica redirección, no un fallo de Python.

12 · Comprobar las tres rutas y diagnosticar

12.1. Repite las pruebas desde Windows

En Windows · PowerShell del portátil

```
curl.exe -i http://IP_ASIGNADA/
curl.exe -i http://IP_ASIGNADA/prueba.php
curl.exe -i http://IP_ASIGNADA/python/
```

Debes recibir 200 en las tres rutas y el texto correspondiente a cada página. Abre también las URL en el navegador. La página HTML debe ofrecer enlaces a PHP y Python; en esas dos páginas la hora debe variar al actualizar.

Ejecuta las órdenes en la pestaña de PowerShell local, donde el indicador empieza por PS. En la página inicial haz clic en Probar PHP y después en Volver al inicio; repite con Probar Python. Guarda para cada respuesta el estado 200 y una línea que identifique HTML, PHP o Python. Así evitas confundir tres respuestas válidas de una misma página con las tres rutas distintas.

12.2. Mira los registros en la Raspberry

En la Raspberry · sesión SSH

```
sudo tail -n 20 /var/log/nginx/access.log
sudo tail -n 20 /var/log/nginx/error.log
sudo journalctl -u ra1-python -n 20 --no-pager
```

access.log registra peticiones, rutas y códigos de respuesta. error.log ayuda a identificar fallos de archivos o de comunicación con los otros programas. journalctl muestra los mensajes de Gunicorn gestionado por systemd. Copia la línea relevante y relaciónala con la hora de tu prueba.

tail -n 20 muestra las últimas veinte líneas y termina; no deja un visor abierto. En access.log busca la ruta, como /prueba.php o /python/, y el número de estado 200. Si error.log no devuelve texto, anota que no había mensajes en el momento de la consulta: no inventes un error.

Para ver la relación entre navegador y registro, abre /python/ otra vez desde el portátil y repite tail sobre access.log. Debe aparecer una petición reciente a esa ruta. Copia una línea; no necesitas entregar todo el registro.

12.3. Prueba después del reinicio

En la Raspberry · sesión SSH

```
sudo reboot
```

Después de pulsar Intro en sudo reboot, la sesión SSH se cerrará y puede aparecer Connection closed. Es lo esperado. Espera a que la Raspberry vuelva a arrancar; un reinicio puede tardar un poco.

En la pestaña local de PowerShell escribe de nuevo ssh USUARIO@IP_ASIGNADA con tus valores y pulsa Intro. Si aún no conecta, espera y vuelve a intentar; si persiste, revisa encendido, IP y red con el docente. Cuando vuelva el indicador de la Raspberry, ejecuta:

En la Raspberry · sesión SSH

```
systemctl is-active nginx
systemctl is-active SERVICIO_PHP
systemctl is-active ra1-python
sudo ss -ltnp
```

Sustituye SERVICIO_PHP por el servicio que anotaste. Nginx, PHP-FPM y Python deben estar activos. El puerto 80 será accesible desde la red y el 8000 debe seguir limitado a 127.0.0.1. Repite después las tres pruebas HTTP desde Windows: un servicio activo no sustituye esa comprobación.

Si aparece un error

Si no hay conexión, separa la prueba local de la remota y revisa IP, servicio, puerto y red. Si aparece 403, comprueba permisos y archivo inicial. Si aparece 404, comprueba ruta y nombre del archivo; para Python revisa la barra final de proxy_pass.

Si aparece 502 en PHP, revisa que PHP-FPM esté activo, que el socket exista y que fastcgi_pass utilice su ruta correcta. Si aparece 502 en Python, prueba primero curl http://127.0.0.1:8000/ y mira el registro de ra1-python. Si PHP devuelve 500, revisa la sintaxis y el registro del servicio PHP-FPM.

Para revisar PHP-FPM, ejecuta sudo systemctl status SERVICIO_PHP --no-pager y sudo journalctl -u SERVICIO_PHP -n 30 --no-pager sustituyendo el nombre real. Para Python, ejecuta sudo systemctl status ra1-python --no-pager. Si necesitas comprobar el socket, repite ls -l /run/php/. Anota el error antes de editar cualquier archivo.

Si nginx -t indica duplicate default server, hay dos sitios configurados como predeterminados para el mismo puerto. Revisa sites-enabled. Si Python indica Address already in use, todavía hay otro proceso en 8000. Identifica el proceso con ss y detén solo tu prueba temporal; no mates procesos al azar.

Corrige un único problema y repite la prueba que fallaba. No reinstales todos los paquetes ni cambies permisos a 777 para intentar resolverlo.

13 · Entregar una práctica sencilla y comprobable

13.1. El producto final

Una Raspberry Pi que responde con HTML, PHP y Python a través de Nginx y conserva ese funcionamiento después del reinicio. Mostrar las tres páginas desde tu portátil forma parte de la demostración.

13.2. Completa tu ficha

En el portátil abre tu carpeta RA1-Nginx y el archivo inventario-nginx.md con el Bloc de notas. Copia la plantilla siguiente y sustituye cada indicación entre corchetes por el dato real que anotaste. Conserva los nombres de los campos. Guarda con Ctrl+S. Esta plantilla se escribe en el archivo de entrega, no en la terminal:

Contenido del archivo · inventario-nginx.md en el portátil

```
# Mi servidor web
Alumno: [tu nombre]
Hostname e IP fija: [datos del equipo]
Sistema y versión: [Raspberry Pi OS/Debian instalado]
Nginx: [versión]
PHP-FPM: [servicio y versión de la página PHP]
Socket PHP: [ruta real]
Python y Gunicorn: [versiones instaladas]
Sitio Nginx: /etc/nginx/sites-available/ra1-web
HTML y PHP: /var/www/ra1-web
Aplicación Python: /opt/ra1-python/app.py
Servicio Python: ra1-python.service
URL HTML: [http://tu-IP/]
URL PHP: [http://tu-IP/prueba.php]
URL Python: [http://tu-IP/python/]
Resultado tras reiniciar: [lo que has comprobado]
```

Para consultar la versión de Gunicorn, ejecuta /opt/ra1-python/.venv/bin/gunicorn --version. Las versiones de bibliotecas Python están en /opt/ra1-python/requirements.txt. No copies la IP de las imágenes ni guardes contraseñas.

13.3. Guarda las evidencias

En comprobaciones-nginx.txt guarda: el resultado de nginx -t; el estado de los tres servicios; las respuestas HTTP con cabecera y texto identificable de HTML, PHP y Python desde Windows; la escucha en 80 y 127.0.0.1:8000; y las tres pruebas HTTP después del reinicio.

Abre comprobaciones-nginx.txt en el portátil. Para organizarlo, repite este pequeño modelo por cada prueba. Pega las salidas de tu propio equipo y guarda con Ctrl+S:

Contenido del archivo · Modelo para comprobaciones-nginx.txt

```
PRUEBA: [qué vas a comprobar]
DÓNDE: [Raspberry por SSH o PowerShell del portátil]
COMANDO: [orden con tus valores reales]
SALIDA: [líneas relevantes de tu resultado]
RESULTADO: [una frase que explique lo observado]
```

Por ejemplo, una prueba de Python desde Windows debe indicar que curl.exe recibió 200 y que el cuerpo contiene Python funciona en mi Raspberry. Después del reinicio crea un bloque nuevo para cada URL; no reutilices la salida anterior como si fuera una prueba nueva.

Añade una línea de access.log que corresponda a una de tus peticiones. Para cada prueba escribe el comando, la salida relevante y una frase sobre lo que demuestra. Si algo queda pendiente, indica el resultado observado y el error; no escribas que funciona sin comprobarlo.

Has terminado cuando…

Puedes abrir las tres páginas después del reinicio y explicar el recorrido: navegador → Nginx → archivo HTML, PHP-FPM o Gunicorn. Los dos archivos permiten identificar tu equipo y repetir tus pruebas. Entrega esos archivos por el medio indicado por el docente; el código y la configuración permanecen en la Raspberry.

Esta práctica utiliza HTTP dentro de la red del aula. HTTPS, dominios, bases de datos y publicación en Internet se trabajarán en prácticas posteriores.

Documentación de consulta

Nginx · Guía de inicio: https://nginx.org/en/docs/beginners_guide.html

Nginx · Proxy HTTP: https://nginx.org/en/docs/http/ngx_http_proxy_module.html

Nginx · FastCGI: https://nginx.org/en/docs/http/ngx_http_fastcgi_module.html

Debian · Organización de Nginx: https://wiki.debian.org/Nginx/DirectoryStructure

PHP · PHP-FPM: https://www.php.net/manual/en/install.fpm.php

Python · Entornos venv: https://docs.python.org/3/library/venv.html

Flask · Gunicorn: https://flask.palletsprojects.com/en/stable/deploying/gunicorn/

Gunicorn · Despliegue: https://gunicorn.org/deploy/